Ex-no.:14
Date:

Write a code using RAW sockets to implement packet sniffing.

Aim:

To write a program using RAW sockets to capture and display network packets passing through the network interface.

Algorithm:

- Start the program.
- Create a RAW socket using socket(AF_PACKET, SOCK_RAW.)
- Bind the socket to the required network interface.
- Continuously receive packets from the network.
- Extract the Ethernet header from each packet.
- Identify the protocol type, such as IPV4, ARP, or IPV6.
- For IPV4 packets, extract the source and destination IP addresses.
- Display the packet information.
- Repeat until the user terminates the program.
- Close the socket.

Program:

```
import socket
import struct
# Create a RAW socket
raw_socket = socket.socket(
    socket.AF_PACKET,
    socket.SOCK_RAW,
    socket.ntohs(3)
)
print("Packet Sniffer started...")
print("Waiting for packets... \n")
try: while True:
# Receive packet
packet, address = raw_socket.recvfrom
                  (65535)
# Ethernet header is first 14 bytes
ethernet_header = packet[:14]
# unpack Ethernet header.
dest_mac, src_mac, protocol = struct
                              .unpack(
"! 6s 6s H", ethernet_header.
print("source IP :", src_ip)
print("Destination IP:", dest_ip)
print("TTL :", ttl)
print("IP Protocol :", proto.)
else:
    print("Packet Type : other")
except keyboardInterrupt:
    print("\n Packet Sniffing stopped").
finally:
    raw_socket.close()
```

Result: Thus, a packet-sniffing program using RAW sockets was successfully implemented. The program captures network packets and displays important information such as source MAC address, protocol, source IP address, destination IP address, and TTL.

Sample output

Packet Sniffer Started
Waiting for packets...
Source MAC : 3C : 52 : 82 : 11 : 22 : 33
Destination MAC : FF : FF : FF : FF : FF : FF
Protocol : 0x806
Packet Type : ARP

Source MAC : 3C : 52 : 82 : 11 : 22 : 33
Destination MAC : 84 : 16 : A9 : 44 : 55 : 66
Protocol : 0x800
Source IP : 192 . 168 . 1 . 10
Destination IP : 192 . 168 . 1 . 1
TTL : 64
IP Protocol : 6

Source MAC : 84 : 16 : A9 : 44 : 55 : 66
Destination MAC : 3C : 52 : 82 : 11 : 22 : 33
Protocol = 0x800
Source IP : 192 . 168 . 1 . 1
Destination IP : 192 . 168 . 1 . 10
TTL : 64
IP Protocol : 6